

Rancang Bangun Aplikasi Playlist Musik
Berbasis CLI Menggunakan Struktur Data *Double Linked List* dan *Stack*

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

PRATAMA FAHRIEL SANJAYA - J0403251053

MUHAMMAD HARIS - J0403251141

RONY WIJAYA - J0403251047

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah Swt. atas segala rahmat dan karunia-Nya sehingga laporan akhir *Project-Based Learning* (PBL) mata kuliah Algoritma dan Struktur Data (TPL2106) ini dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pengerjaan proyek “Rancang Bangun Aplikasi Playlist Musik Berbasis CLI Menggunakan Struktur Data *Double Linked List* dan *Stack*” yang dikembangkan oleh Kelompok 13. Proyek ini mengimplementasikan konsep-konsep yang telah dipelajari selama perkuliahan, khususnya struktur data *Double Linked List* dan *Stack*, beserta algoritma pencarian dan pengurutan, dalam sebuah aplikasi nyata berbasis Python.

Penulis menyadari bahwa keberhasilan penyelesaian proyek dan laporan ini tidak terlepas dari dukungan dan bimbingan berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dr. Shelvie Nidya Neyman, S.Kom., M.Si. selaku dosen pengampu mata kuliah Algoritma dan Struktur Data yang telah memberikan bimbingan, arahan, dan ilmu yang sangat bermanfaat selama perkuliahan berlangsung.
2. Dr. Sofiyanti Indriasari, S.Kom., M.Kom. selaku dosen mata kuliah Algoritma dan Struktur Data yang turut memberikan arahan dan masukan dalam proses pembelajaran.
3. Keluarga dan teman-teman yang telah memberikan dukungan moral serta semangat selama proses pengerjaan proyek dan penyusunan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun dari pembaca sangat penulis harapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari implementasi struktur data dan algoritma dalam pengembangan perangkat lunak.

Bogor, Juni 2026

Kelompok 13

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Proyek.....	2
1.3 Ruang Lingkup	2
BAB II ANALISIS DAN PERANCANGAN.....	3
2.1 Deskripsi Sistem.....	3
2.2 Struktur Data yang Digunakan	3
2.3 Algoritma yang Digunakan	4
2.4 Diagram Alir Program (Flowchart)	5
2.5 Struktur	6
BAB III IMPLEMENTASI PROGRAM.....	7
3.1 Tampilan Menu Utama.....	7
3.2 Fitur <i>Create</i>	8
3.3 Fitur Read	9
3.4 Fitur Update.....	10
3.5 Fitur Delete.....	11
3.6 Fitur Searching	13
3.7 Fitur Sorting.....	14
3.8 Penyimpanan Data (File Handling)	16
3.9 Validasi Input dan Error Handling	18
BAB IV PENGUJIAN DAN ANALISIS	19
4.1 Skenario Pengujian.....	19
4.1.1 Hasil Pengujian Fungsionalitas CRUD	19
4.1.2 Hasil Pengujian Fitur Playback.....	20
4.1.3 Hasil Pengujian Searching dan Sorting.....	20
4.1.4 Hasil Pengujian History Stack	21
4.1.5 Hasil Pengujian File Handling dan Persistensi Data.....	21
4.2 Kelebihan Sistem	21
4.2.1 Arsitektur Struktur Data.....	21
4.2.2 Fitur dan Fungsionalitas Sistem	22

4.2.3	User Experience	22
4.3	Keterbatasan Sistem.....	23
4.3.1	Aspek Teknis.....	23
4.4.2	Aspek Fungsional.....	24
4.4.3	Batasan Performa	24
BAB V. KENDALA DAN SOLUSI.....		25
BAB VI. PEMBAGIAN TUGAS ANGGOTA		26
BAB VII. KESIMPULAN DAN SARAN		27
DAFTAR PUSTAKA		29
LAMPIRAN.....		30
Lampiran A. Link GitHub.....		30
Lampiran B. Screenshot Program		30
Lampiran C. Struktur Folder Proyek.....		30
Lampiran D. Commit History GitHub		31
Lampiran E. Source Code Utama		32

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital telah mengubah cara manusia mengonsumsi musik secara fundamental. Platform streaming modern seperti Spotify dan Apple Music kini mengelola jutaan lagu dengan fitur navigasi yang kompleks, termasuk perpindahan lagu ke depan dan ke belakang, fitur acak (*shuffle*), pengulangan (*loop*), serta riwayat pemutaran. Di balik antarmuka yang tampak sederhana tersebut, terdapat fondasi struktur data dan algoritma yang dirancang secara cermat untuk memenuhi kebutuhan operasi tersebut secara efisien.

Dalam konteks pembelajaran Algoritma dan Struktur Data, perancangan sebuah aplikasi pemutar musik berbasis terminal (*Command Line Interface/CLI*) merupakan studi kasus yang sangat relevan. Pemilihan struktur data yang tepat akan secara langsung menentukan efisiensi dan kemampuan fungsional sistem. Goodrich, Tamassia, dan Goldwasser (2013) menegaskan bahwa pemilihan struktur data yang sesuai dengan karakteristik operasi yang akan dilakukan adalah kunci keberhasilan perancangan perangkat lunak yang efisien.

Salah satu tantangan utama dalam pengembangan pemutar musik adalah mengelola navigasi lagu secara dua arah (maju dan mundur) dengan kompleksitas waktu yang optimal. Struktur data *Double Linked List* (DLL) terbukti ideal untuk skenario ini karena setiap simpul (node) memiliki pointer ke simpul berikutnya (*next*) sekaligus simpul sebelumnya (*prev*), sehingga operasi perpindahan lagu dapat dilakukan dalam waktu konstan $O(1)$. Di sisi lain, pengelolaan riwayat pemutaran membutuhkan struktur Stack yang mengimplementasikan prinsip LIFO (*Last In, First Out*), memungkinkan pengguna mengakses kembali lagu terakhir yang diputar dalam waktu $O(1)$ tanpa *overhead* iterasi.

Berdasarkan permasalahan tersebut, Kelompok 13 mengembangkan Aplikasi Playlist Musik Berbasis Terminal menggunakan bahasa pemrograman Python. Aplikasi ini mengimplementasikan Double Linked List sebagai struktur data utama untuk pengelolaan playlist dan Stack untuk manajemen riwayat pemutaran, didukung library *pygame* untuk pemutaran audio nyata dan *library* *mutagen* untuk ekstraksi metadata file .mp3. Fitur yang disediakan meliputi CRUD (*Create, Read, Update, Delete*), pencarian, dan pengurutan lagu, serta penyimpanan data persisten menggunakan file CSV.

1.2 Tujuan Proyek

Adapun tujuan dari pengembangan proyek ini adalah sebagai berikut:

- 1 Membuat aplikasi pemutar musik berbasis terminal (CLI) menggunakan bahasa pemrograman Python.
- 2 Mengimplementasikan struktur data Double Linked List untuk mengelola urutan lagu dalam playlist secara efisien.
- 3 Mengimplementasikan struktur data Stack untuk mengelola riwayat pemutaran lagu.
- 4 Mengembangkan fitur CRUD, pencarian (searching), dan pengurutan (sorting) pada data lagu.

1.3 Ruang Lingkup

Ruang lingkup pengembangan aplikasi ini mencakup:

- 1 Platform: CLI (Console/Terminal), berbasis teks tanpa antarmuka grafis.
- 2 Bahasa Pemrograman: Python 3.
- 3 Penyimpanan Data: File CSV (playlist.csv) untuk persistensi data lagu.
- 4 Fitur CRUD: Tambah, tampilkan, perbarui, dan hapus data lagu.
- 5 Searching: Pencarian lagu berdasarkan judul atau nama artis.
- 6 Sorting: Mengurutkan lagu berdasarkan judul (A-Z) atau durasi.
- 7 Struktur Data: Double Linked List sebagai struktur utama dan Stack untuk manajemen riwayat pemutaran.
- 8 Pemutaran Musik: Kontrol pemutaran nyata menggunakan library pygame.mixer (Play, Next, Previous, Pause, Resume, Shuffle, Loop).
- 9 Metadata Lagu: Ekstraksi otomatis judul, artis, dan durasi dari file .mp3 menggunakan library mutagen.

BAB II

ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Aplikasi Playlist Musik Berbasis Terminal adalah sebuah sistem manajemen dan simulasi pemutar musik yang dibangun di atas antarmuka *Command Line Interface* (CLI) menggunakan Python. Sistem ini memungkinkan pengguna untuk mengelola koleksi lagu secara lengkap, mulai dari menambahkan lagu baru dengan ekstraksi metadata otomatis dari file .mp3, memperbarui informasi lagu, hingga menghapus lagu dari *playlist*.

Saat program dijalankan, sistem secara otomatis memindai folder 'Music' atau membaca file 'playlist.csv' yang telah tersedia untuk memuat data lagu ke dalam memori. Metadata lagu seperti judul, artis, dan durasi diekstrak secara otomatis dari file .mp3 menggunakan *library* mutagen. Data tersebut disimpan dalam struktur *Double Linked List* agar navigasi antar lagu dapat dilakukan secara efisien ke dua arah. Pengguna dapat mengontrol pemutaran audio secara nyata menggunakan *library* pygame.mixer melalui perintah *Play*, *Next*, *Previous*, *Pause*, *Resume*, *Shuffle*, dan *Loop*, sementara riwayat pemutaran dicatat menggunakan *Stack*. Selain itu, tersedia fitur pencarian dan pengurutan lagu untuk memudahkan pengguna menemukan dan mengorganisir koleksi musiknya.

2.2 Struktur Data yang Digunakan

Berikut adalah tabel struktur data yang diimplementasikan beserta fungsi masing-masing:

Tabel 2.1 Struktur Data

Struktur Data	Fungsi
Double Linked List	Struktur data utama untuk menyimpan dan mengelola urutan lagu dalam playlist. Mendukung navigasi dua arah (Next/Previous) dengan kompleksitas $O(1)$ serta operasi CRUD yang efisien tanpa pergeseran elemen.
Stack	Mengelola riwayat pemutaran lagu (Playback History) dengan prinsip LIFO. Digunakan untuk mengakses kembali lagu terakhir yang diputar, terutama saat mode Shuffle aktif, dengan kompleksitas $O(1)$.

Struktur Data	Fungsi
Array/List Python	Digunakan sebagai struktur pendukung dalam proses pengurutan (sorting). Node-node DLL dikonversi sementara ke List Python untuk diurutkan, kemudian dimasukkan kembali ke dalam DLL.

Alasan Pemilihan Struktur Data

Double Linked List (DLL) dipilih sebagai struktur utama karena secara presisi memodelkan karakteristik linear pemutaran musik. Berbeda dengan *Binary Search Tree* (BST) atau AVL Tree yang dioptimalkan untuk pencarian hierarkis, DLL memungkinkan perpindahan ke node (lagu) berikutnya atau sebelumnya dengan kompleksitas waktu $O(1)$ melalui modifikasi *pointer next* dan *prev*. Manipulasi data seperti penambahan dan penghapusan lagu juga jauh lebih efisien dibandingkan Array karena tidak memerlukan pergeseran elemen (*shifting*). Miller dan Ranum (2011) menjelaskan bahwa Linked List memberikan fleksibilitas alokasi memori dinamis yang sangat sesuai untuk kasus di mana ukuran data tidak diketahui pada waktu kompilasi. Selain itu, fitur pencarian pada aplikasi ini diimplementasikan langsung melalui traversal DLL secara *Sequential Search*, sehingga tidak diperlukan struktur data tambahan seperti *Dictionary*.

Stack dipilih untuk mengelola riwayat pemutaran karena secara alami memodelkan perilaku riwayat yang bersifat linear berdasarkan garis waktu. Prinsip LIFO memastikan bahwa lagu terakhir yang diputar selalu dapat diakses dalam waktu $O(1)$ melalui operasi pop, tanpa memerlukan iterasi. Cormen et al. (2022) menekankan bahwa *Stack* adalah pilihan optimal untuk skenario akses berurutan terbalik, seperti riwayat navigasi atau *undo/redo*.

2.3 Algoritma yang Digunakan

Searching

Algoritma: Sequential Search (Linear Search).

Cara kerja: Program melakukan traversal dari node head DLL secara berurutan. Setiap node diperiksa apakah atribut title atau artist-nya mengandung keyword yang dicari (*case-insensitive* menggunakan `.lower()`). Node yang cocok dikumpulkan dalam list hasil dan dikembalikan ke pengguna.

Alasan: Sequential Search dipilih karena data playlist bersifat tidak terurut secara default dan ukurannya relatif kecil (koleksi musik personal). Struktur DLL juga memang dirancang untuk traversal linear, sehingga Sequential Search menjadi pilihan yang natural dan mudah diimplementasikan. Kompleksitasnya $O(n)$ dimana n adalah jumlah lagu dalam playlist.

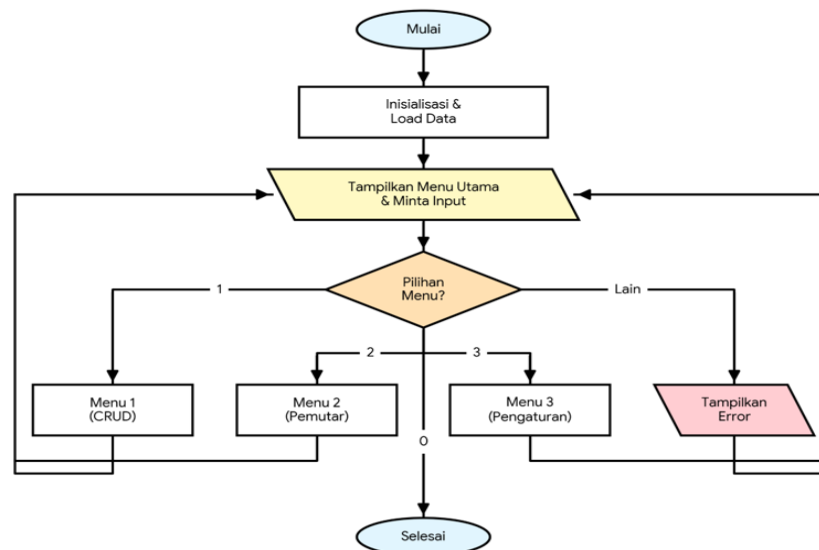
Sorting

Algoritma: Bubble Sort (untuk pengurutan berdasarkan judul A-Z) dan perbandingan nilai integer (untuk pengurutan berdasarkan durasi).

Cara kerja: Node-node dalam DLL dikonversi sementara ke dalam List Python, kemudian diurutkan menggunakan Bubble Sort dengan perbandingan string atau nilai numerik durasi. Setelah selesai, List dikonversi kembali ke DLL.

Alasan: Bubble Sort dipilih karena implementasinya sederhana dan cocok untuk ukuran dataset kecil hingga menengah (koleksi musik personal). Kompleksitasnya $O(n^2)$ pada kasus terburuk dan $O(n)$ pada data yang hampir terurut.

2.4 Diagram Alir Program (Flowchart)



Gambar 2.1 Flowchart

2.5 Struktur

Berikut adalah file-file yang dibuat dalam pengembangan aplikasi ini:

1. `Playlist_Musik.py` adalah file utama yang mengintegrasikan seluruh modul, menjalankan program, menampilkan menu interaktif, serta mengimplementasikan Double Linked List dan kontrol pemutaran menggunakan `pygame.mixer`.
2. `history_stack.py` adalah implementasi kelas Stack untuk manajemen riwayat pemutaran lagu (LIFO).
3. `search_sort.py` adalah modul pencarian lagu (Sequential Search berdasarkan judul dan artis) serta pengurutan lagu (Bubble Sort).
4. `data_handler.py` adalah modul baca/tulis file CSV untuk persistensi data playlist, termasuk ekstraksi metadata `.mp3` menggunakan library `mutagen`.
5. `playlist.csv` adalah file penyimpanan data lagu secara permanen.

BAB III

IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

Program Playlist Musik Berbasis Terminal diawali dengan menampilkan menu utama sebagai pusat navigasi pengguna untuk mengakses seluruh fitur yang tersedia. Saat aplikasi dijalankan, sistem secara otomatis memuat data lagu melalui fungsi `load_music_folder()`, yang bertugas memindai folder Music dan mendeteksi seluruh file audio berformat MP3 di dalamnya.

Setiap file yang ditemukan kemudian diproses untuk mengambil informasi metadata seperti judul lagu, nama artis, dan durasi pemutaran. Ekstraksi metadata dilakukan secara otomatis menggunakan library Mutagen, sehingga informasi lagu dapat langsung ditampilkan dan dikelola tanpa perlu input manual dari pengguna. Setelah metadata berhasil diperoleh, data lagu dimasukkan ke dalam struktur playlist yang digunakan sistem untuk mendukung berbagai operasi pengelolaan dan pemutaran musik. Struktur menu pada aplikasi dirancang secara terorganisir agar pengguna dapat mengakses berbagai fitur sesuai fungsinya dengan mudah. Setiap kelompok menu memiliki tanggung jawab yang berbeda dalam pengelolaan dan pengoperasian playlist musik.

```
1  ===== MENU UTAMA =====
2  1. Menu Playlist (CRUD)
3  2. Menu Lagu
4  3. Menu Utility
5  0. Keluar
6
7  Pilih menu:
```

Gambar.3.1 Menu Utama

1. Menu Playlist (CRUD)

Menu ini digunakan untuk mengelola data lagu, meliputi penambahan lagu baru (Create), menampilkan daftar lagu (Read), memperbarui informasi lagu (Update), serta menghapus lagu dari playlist (Delete).

2. Menu Lagu

Menu ini menyediakan berbagai kontrol pemutaran yang memungkinkan pengguna mengelola proses playback secara langsung. Fitur yang tersedia meliputi memutar lagu (Play), berpindah ke lagu berikutnya (Next), kembali ke lagu sebelumnya (Previous), menjeda pemutaran (Pause), melanjutkan pemutaran (Resume), dan menghentikan lagu (Stop).

3. Menu Utility

Menu ini berisi fitur-fitur pendukung untuk membantu pengguna mengelola koleksi musik secara lebih efektif. Fasilitas yang tersedia mencakup pencarian lagu (Searching), pengurutan data lagu (Sorting), melihat riwayat pemutaran (Playback History), serta menyimpan data playlist ke penyimpanan secara permanen.

3.2 Fitur *Create*

Fitur *Create* digunakan untuk menambahkan data lagu baru ke dalam *playlist* yang dikelola sistem. Melalui fitur ini, pengguna dapat memperluas koleksi lagu yang tersedia untuk diputar dan dikelola melalui aplikasi.

Saat program berjalan, sistem memindai folder Music dan mendeteksi seluruh file audio berformat MP3 yang ada di dalamnya. Setiap file yang dipilih akan diproses dan ditambahkan ke playlist setelah metadata lagu berhasil diekstrak.

```
1 def add_song(self, id_song, title, artist, duration, file_path):
2     if title.strip() == "" or artist.strip() == "":
3         print("Judul dan artis tidak boleh kosong!")
4         return
5
6     current = self.head
7     while current:
8         if current.id == id_song:
9             print("ID sudah digunakan!")
10            return
11            current = current.next
12
13     new_song = SongNode(id_song, title, artist, duration, file_path)
14
15     if self.head is None:
16         self.head = new_song
17         self.tail = new_song
18     else:
19         self.tail.next = new_song
20         new_song.prev = self.tail
21         self.tail = new_song
22
23     print("Lagu berhasil ditambahkan!")
```

Gambar 3.2 Fitur Create

Penjelasan Algoritma

Proses penambahan lagu dilakukan melalui beberapa tahapan untuk memastikan data yang dimasukkan valid dan tidak menimbulkan inkonsistensi pada struktur data.

1. Validasi Input: Sistem memeriksa data yang dimasukkan pengguna. Atribut penting seperti judul lagu dan nama artis wajib terisi. Jika ada data yang kosong, proses penambahan dibatalkan dan pengguna diminta melengkapi informasi yang diperlukan.
2. Pemeriksaan Duplikasi Data: Setelah data dinyatakan valid, program melakukan traversal pada Double Linked List dari node head hingga tail untuk memastikan tidak ada lagu dengan ID yang sama. Karena seluruh node diperiksa secara berurutan, proses ini memiliki kompleksitas waktu $O(n)$.
3. Pembuatan Node Baru: Jika tidak ditemukan duplikasi, sistem membuat objek `SongNode` baru yang berisi informasi lagu seperti ID, judul, artis, durasi, dan lokasi file audio.
4. Penambahan Node ke Double Linked List: Node yang telah dibuat dimasukkan ke dalam DLL. Jika playlist masih kosong, node baru ditetapkan sebagai head sekaligus tail. Jika playlist sudah berisi data, node baru ditambahkan di bagian akhir (tail) dengan memperbarui pointer next dan prev agar hubungan antar node tetap terjaga.

Kompleksitas Waktu: $O(n)$, berasal dari tahap pemeriksaan duplikasi yang memerlukan traversal seluruh node. Sementara proses pembuatan dan penambahan node ke tail DLL dapat dilakukan dalam waktu konstan $O(1)$.

3.3 Fitur Read

Fitur Read berfungsi untuk menampilkan seluruh data lagu yang tersimpan dalam playlist. Melalui fitur ini, pengguna dapat melihat informasi lengkap setiap lagu yang telah ditambahkan ke sistem, meliputi ID lagu, judul lagu, nama artis, dan durasi pemutaran. Data disajikan secara terstruktur agar pengguna dapat mengidentifikasi dan mengelola koleksi musik dengan mudah.

```
1  def show_playlist(self):
2      if self.head is None:
3          print("\nPlaylist kosong.")
4          return
5
6      current = self.head
7      print("\n=== PLAYLIST ===")
8      while current:
9          print(f"{current.id}. {current.title} - {current.artist} ({current.duration})")
10         current = current.next
```

Gambar 3.3 Fitur Read

Pada implementasinya, sistem melakukan traversal pada DLL mulai dari node head hingga tail. Setiap node yang dikunjungi ditampilkan beserta atributnya. Jika playlist masih kosong, sistem menampilkan pesan bahwa belum ada lagu yang tersimpan.

Penjelasan Algoritma

1. Pemeriksaan Kondisi Playlist: Sistem memeriksa apakah playlist memiliki data. Jika nilai head bernilai None, playlist dianggap kosong dan program menampilkan pesan bahwa tidak ada lagu yang tersedia.
2. Traversal Double Linked List: Jika playlist berisi data, sistem memulai traversal dari node head, lalu pointer bergerak ke node berikutnya menggunakan referensi next hingga mencapai node terakhir (tail).
3. Menampilkan Informasi Lagu: Pada setiap node yang dikunjungi, program menampilkan informasi lagu seperti ID, judul, nama artis, dan durasi pemutaran.
4. Mengakhiri Traversal: Proses berakhir ketika pointer mencapai nilai None, menandakan seluruh data lagu telah berhasil ditampilkan.

Kompleksitas Waktu : $O(n)$, karena sistem harus mengunjungi setiap node satu per satu untuk menampilkan seluruh informasi lagu.

3.4 Fitur Update

Fitur Update digunakan untuk mengubah informasi lagu yang telah tersimpan dalam playlist. Pengguna dapat memperbarui data lagu jika terdapat kesalahan informasi atau perubahan metadata yang perlu disesuaikan. Informasi yang dapat diperbarui meliputi judul lagu, nama artis, durasi, maupun lokasi file (file path).

```
1 def edit_song(self, id_song, new_title=None, new_artist=None, new_duration=None, new_file_path=None):
2     current = self.head
3
4     while current:
5         if current.id == id_song:
6             if new_title and new_title.strip() != "":
7                 current.title = new_title.strip()
8             if new_artist and new_artist.strip() != "":
9                 current.artist = new_artist.strip()
10            if new_duration and new_duration.strip() != "":
11                current.duration = new_duration.strip()
12            if new_file_path and new_file_path.strip() != "":
13                current.file_path = new_file_path.strip()
14            print("Lagu berhasil diedit!")
15            return
16        current = current.next
17
18     print("ID tidak ditemukan!")
```

Gambar 3.4 Fitur Update

Proses pembaruan dilakukan dengan mencari lagu berdasarkan ID yang dipilih pengguna. Setelah data ditemukan, sistem memperbarui atribut yang diinginkan tanpa mengubah informasi lain yang tetap dipertahankan.

Penjelasan Algoritma

1. Pencarian Data Lagu: Sistem melakukan traversal pada DLL dari node head hingga menemukan node dengan ID yang sesuai dengan masukan pengguna.
2. Validasi Keberadaan Data: Jika node dengan ID yang dicari tidak ditemukan, sistem menampilkan pesan bahwa data tidak tersedia dan proses pembaruan dibatalkan.
3. Pembaruan Informasi Lagu: Jika node ditemukan, sistem memperbarui atribut yang diisi pengguna, seperti judul, artis, durasi, atau lokasi file.
4. Partial Update: Sistem menerapkan mekanisme partial update, yaitu hanya atribut yang diberi nilai baru yang diperbarui. Atribut yang dibiarkan kosong tetap mempertahankan nilai sebelumnya sehingga tidak terjadi kehilangan data.

Kompleksitas Waktu: $O(n)$, disebabkan oleh proses pencarian node yang dilakukan secara traversal dari head hingga data ditemukan atau mencapai akhir daftar.

3.5 Fitur Delete

Fitur Delete digunakan untuk menghapus lagu dari playlist dan menghentikan pemutaran jika lagu tersebut sedang diputar.

```
1  def delete_song(self, id_song):
2      current = self.head
3      id_target = str(id_song)
4
5      while current:
6          if str(current.id) == id_target:
7              if current == self.head:
8                  self.head = current.next
9                  if self.head:
10                     self.head.prev = None
11             else:
12                 self.tail = None
13             elif current == self.tail:
14                 self.tail = current.prev
15                 if self.tail:
16                     self.tail.next = None
17             else:
18                 current.prev.next = current.next
19                 current.next.prev = current.prev
20
21             # Reset current jika lagu yang sedang diputar dihapus
22             if self.current == current:
23                 self.current = None
24                 pygame.mixer.music.stop()
25
26             print(f"Lagu dengan ID {id_song} berhasil dihapus!")
27             return
28
29         current = current.next
30
31     print(f"ID {id_song} tidak ditemukan!")
```

Gambar 3.5 Fitur Delete

Algoritma Penghapusan Node pada Double Linked List

Proses penghapusan data pada Double Linked List dilakukan dengan mencari node yang sesuai terlebih dahulu, kemudian memperbarui hubungan antar pointer agar struktur list tetap berjalan dengan benar. Karena setiap node memiliki pointer prev dan next, proses penghapusan dapat dilakukan tanpa perlu menggeser elemen seperti pada struktur array.

Tahapan implementasi algoritma penghapusan node adalah sebagai berikut:

1. Traversal Pencarian Node Target: Sistem melakukan penelusuran dari head hingga menemukan node yang memiliki data sesuai dengan ID atau parameter yang ingin dihapus.
2. Penghapusan Node Head: Jika node yang akan dihapus berada pada posisi head, maka pointer head akan dipindahkan ke node berikutnya. Setelah itu, pointer prev pada head baru diatur menjadi None untuk menandakan bahwa node tersebut menjadi awal list.
3. Penghapusan Node Tail: Jika node target berada pada posisi tail, maka pointer tail akan diarahkan ke node sebelumnya. Selanjutnya, pointer next pada tail baru diatur menjadi None.
4. Penghapusan Node Tengah: Jika node berada di antara head dan tail, maka hubungan antar node diperbaiki dengan menghubungkan prev.next ke next dan next.prev ke prev, sehingga node target dapat dilepas dari struktur Double Linked List.
5. Reset Current Pointer: Apabila lagu yang sedang diputar merupakan node yang dihapus, sistem akan menghentikan proses pemutaran musik dan melakukan reset terhadap pointer current.

Kompleksitas Waktu

Proses penghapusan node memiliki kompleksitas waktu $O(n)$ karena sistem perlu melakukan traversal terlebih dahulu untuk menemukan node yang akan dihapus.

Keuntungan Penggunaan Double Linked List untuk Operasi Delete. Penggunaan Double Linked List memberikan keuntungan karena setiap node memiliki pointer menuju node sebelumnya (prev) dan node berikutnya (next). Dengan adanya pointer tersebut, proses penghapusan node dapat dilakukan secara lebih efisien karena tidak membutuhkan traversal tambahan untuk kembali ke node sebelumnya, berbeda dengan Single Linked List yang hanya memiliki pointer satu arah.

3.6 Fitur Searching

Fitur searching memungkinkan pengguna mencari lagu berdasarkan judul atau nama artis menggunakan algoritma Sequential Search.

Implementasi Sequential Search:

```
1  def search_title(playlist, keyword):
2      """Cari lagu berdasarkan kata kunci di judul (case-insensitive)."""
3      results = []
4      curr = playlist.head
5      while curr:
6          if keyword.lower() in curr.title.lower():
7              results.append(curr)
8              curr = curr.next
9      return results
10
11 def search_artist(playlist, keyword):
12     """Cari lagu berdasarkan kata kunci di nama artis (case-insensitive)."""
13     results = []
14     curr = playlist.head
15     while curr:
16         if keyword.lower() in curr.artist.lower():
17             results.append(curr)
18             curr = curr.next
19     return results
```

Gambar 3.6 Fitur Search

Algoritma Sequential Search

Algoritma Sequential Search digunakan untuk melakukan proses pencarian data lagu dengan cara memeriksa setiap elemen secara berurutan pada struktur Double Linked List. Proses pencarian dimulai dari node pertama (head) hingga node terakhir (tail) untuk menemukan data yang sesuai dengan kata kunci yang diberikan pengguna.

Tahapan implementasi algoritma Sequential Search adalah sebagai berikut:

1. Inisialisasi List Hasil: Sistem membuat sebuah list kosong yang digunakan untuk menyimpan data lagu yang berhasil ditemukan selama proses pencarian.
2. Traversal dari Head ke Tail: Proses pencarian dilakukan dengan mengunjungi setiap node pada Double Linked List secara berurutan mulai dari head hingga tail.
3. Perbandingan Data dengan Keyword: Setiap data pada node akan dibandingkan dengan kata kunci pencarian menggunakan metode case-insensitive dengan mengubah seluruh karakter menjadi huruf kecil menggunakan fungsi `.lower()`, sehingga pencarian tidak dipengaruhi oleh perbedaan huruf kapital.

4. Penyimpanan Data yang Sesuai: Node yang memiliki kecocokan dengan keyword akan dimasukkan ke dalam list hasil pencarian.
5. Pengembalian Hasil Pencarian: Setelah seluruh node selesai diperiksa, sistem mengembalikan seluruh data yang sesuai dengan kata kunci yang diberikan.

Kompleksitas Waktu

Algoritma Sequential Search memiliki kompleksitas waktu sebesar $O(n)$, dengan n merupakan jumlah data lagu yang terdapat dalam playlist. Hal ini disebabkan karena algoritma harus melakukan pengecekan terhadap setiap node hingga menemukan seluruh data yang sesuai.

3.7 Fitur Sorting

Fitur sorting mengurutkan playlist berdasarkan judul (A-Z) atau durasi (pendek ke panjang) menggunakan algoritma Bubble Sort.

Implementasi Bubble Sort - Pengurutan Judul:

```

1  def sort_title_asc(playlist):
2      """Urutkan playlist berdasarkan judul lagu secara ascending (A-Z) menggunakan bubble sort."""
3      if not playlist.head:
4          return
5
6      swapped = True
7      while swapped:
8          swapped = False
9          curr = playlist.head
10
11         while curr and curr.next:
12             if curr.title > curr.next.title: # Perbandingan lexicographical
13                 # Simpan next_node karena akan diubah
14                 next_node = curr.next
15
16                 # Proses pertukaran node di doubly linked list
17                 prev = curr.prev
18                 if prev:
19                     prev.next = next_node
20                 else:
21                     playlist.head = next_node
22
23                 curr.next = next_node.next
24                 next_node.prev = prev
25                 next_node.next = curr
26                 curr.prev = next_node
27
28                 if curr.next:
29                     curr.next.prev = curr
30                 else:
31                     playlist.tail = curr
32
33                 swapped = True
34                 curr = curr.next
35             else:
36                 curr = curr.next
37         print("Diurutkan judul A-Z")
38         playlist.show_playlist()

```

Gambar 3.7 Fitur Sort- Judul

Algoritma Bubble Sort pada Struktur Double Linked List

Proses pengurutan data menggunakan algoritma Bubble Sort pada Double Linked List dilakukan dengan beberapa tahapan. Sistem menggunakan variabel penanda (swapped flag) untuk mengetahui

apakah terjadi proses pertukaran data selama iterasi berlangsung. Apabila dalam satu kali perulangan tidak ditemukan pertukaran, maka data dianggap telah berada dalam kondisi terurut dan proses sorting dapat dihentikan.

Tahapan implementasi algoritma Bubble Sort pada Double Linked List adalah sebagai berikut:

1. Flag Pertukaran: Sistem menggunakan variabel **swapped** sebagai indikator apakah terdapat perubahan posisi node pada setiap iterasi. Jika tidak ada pertukaran, proses pengurutan dihentikan karena data sudah tersusun dengan benar.
2. Outer Loop: Perulangan utama dilakukan secara berulang hingga tidak ditemukan adanya pertukaran antar node.
3. Inner Loop: Sistem melakukan traversal mulai dari node head dan membandingkan nilai setiap node dengan node setelahnya.
4. Pertukaran Node: Apabila data pada node saat ini memiliki nilai lebih besar dibandingkan node berikutnya, maka dilakukan proses pertukaran data dengan memperbarui hubungan pointer antar node.
5. Update Pointer Head dan Tail: Setelah proses pertukaran dilakukan, sistem memastikan bahwa pointer head dan tail tetap menunjuk ke node yang benar sehingga struktur Double Linked List tetap valid.

Kompleksitas Algoritma

Kompleksitas waktu algoritma Bubble Sort pada Double Linked List bergantung pada kondisi awal data:

1. Best Case: $O(n)$
Terjadi ketika data sudah berada dalam kondisi terurut sehingga algoritma hanya membutuhkan satu kali traversal untuk memastikan tidak terdapat pertukaran.
2. Average Case: $O(n^2)$
Terjadi pada kondisi umum ketika sebagian besar data masih memerlukan proses pertukaran.
3. Worst Case: $O(n^2)$
Terjadi ketika data berada dalam kondisi terbalik sehingga algoritma harus melakukan jumlah perbandingan dan pertukaran maksimum.

Implementasi Bubble Sort - Pengurutan Durasi:

```

1 def sort_duration_asc(playlist):
2     """Urutkan playlist berdasarkan durasi lagu secara ascending (pendek ke panjang)."""
3
4     def duration_to_sec(dur):
5         try:
6             if '.' in dur:
7                 m, s = map(int, dur.split('.'))
8             elif ':' in dur:
9                 m, s = map(int, dur.split(':'))
10            else:
11                return 0
12            return m*60 + s
13        except (ValueError, AttributeError):
14            return 0
15
16    if not playlist.head:
17        return
18
19    swapped = True
20    while swapped:
21        swapped = False
22        curr = playlist.head
23
24        while curr and curr.next:
25            d1 = duration_to_sec(curr.duration)
26            d2 = duration_to_sec(curr.next.duration)
27
28            if d1 > d2: # Bandingkan dalam detik
29                next_node = curr.next
30
31                # Proses pertukaran (sama seperti sort_title_asc)
32                prev = curr.prev
33                if prev:
34                    prev.next = next_node
35                else:
36                    playlist.head = next_node
37
38                curr.next = next_node.next
39                next_node.prev = prev
40                next_node.next = curr
41                curr.prev = next_node
42
43                if curr.next:
44                    curr.next.prev = curr
45                else:
46                    playlist.tail = curr
47
48                swapped = True
49                curr = curr.next
50            else:
51                curr = curr.next
52
53    print("Diurutkan durasi pendek-panjang")
54    playlist.show_playlist()

```

Gambar 3.8 Fitur Sort- Durasi

3.8 Penyimpanan Data (File Handling)

Data playlist pada sistem disimpan menggunakan format file CSV sebagai media penyimpanan persisten. Penggunaan penyimpanan ini bertujuan agar seluruh data lagu tetap tersimpan dan dapat dimuat kembali meskipun program telah ditutup. Dengan mekanisme tersebut, informasi playlist tidak akan hilang ketika aplikasi dijalankan ulang.

Format Struktur File CSV:

ID,Title,Artist,Duration,FilePath

1,Gemilang,Perunggu,5:00,C:\Music\Perunggu-Gemilang.mp3

2,Tahu Bulat Mantul,ronywijaya825,4:15,C:\Music\Rony - Tahu Bulat Mantul.mp3

Setiap data lagu disimpan dalam beberapa kolom dengan fungsi masing-masing sebagai berikut:

1. ID: Berfungsi sebagai identitas unik untuk setiap data lagu sehingga setiap lagu dapat dibedakan dan dikelola secara spesifik.
2. Title: Menyimpan informasi mengenai judul lagu yang terdapat pada playlist.
3. Artist: Berisi nama artis atau penyanyi dari lagu yang tersimpan.
4. Duration: Menyimpan informasi durasi lagu dengan format waktu menit dan detik (MM:SS).
5. FilePath: Menyimpan lokasi lengkap file MP3 pada perangkat agar sistem dapat mengakses file audio ketika proses pemutaran dilakukan.

Implementasi Save ke CSV:

```
1 def save_to_csv(playlist):
2     """Simpan playlist ke file CSV agar data tidak hilang saat restart aplikasi."""
3     try:
4         data = []
5         curr = playlist.head
6         while curr:
7             data.append([curr.id, curr.title, curr.artist, curr.duration, curr.file_path])
8             curr = curr.next
9
10        with open(CSV_FILE, 'w', newline='', encoding='utf-8') as f:
11            writer = csv.writer(f)
12            writer.writerow(['ID', 'Title', 'Artist', 'Duration', 'FilePath'])
13            writer.writerows(data)
14            print(f"Data disimpan ke {CSV_FILE}")
15    except Exception as e:
16        print(f"Error menyimpan CSV: {e}")
```

Gambar 3.9 Implementasi CSV

Proses penyimpanan:

1. Inisialisasi list kosong untuk data
2. Traversal DLL dari head ke tail mengumpulkan informasi setiap lagu
3. Buka file CSV dalam mode write
4. Tulis header kolom
5. Tulis semua data lagu ke file

Kompleksitas Waktu: $O(n)$ untuk traversal + $O(n)$ untuk penulisan = $O(n)$

Implementasi Load dari CSV:

Data CSV dimuat secara otomatis saat program dijalankan atau bisa dimuat secara manual oleh pengguna. Program juga menyediakan fungsi `load_music_folder()` untuk scan folder Music.

3.9 Validasi Input dan Error Handling

Program mengimplementasikan error handling yang komprehensif untuk menangani berbagai kondisi error.

Tabel 3.1 Tabel Error Handling

Kasus Error	Penanganan
Input huruf pada pilihan menu angka	Program meminta input ulang dengan pesan "Menu tidak valid!"
Judul atau artis kosong saat tambah lagu	Pesan "Judul dan artis tidak boleh kosong!" ditampilkan
ID lagu sudah digunakan	Pesan "ID sudah digunakan!" dan lagu tidak ditambahkan
ID lagu tidak ditemukan saat edit/delete	Pesan "ID tidak ditemukan!" dan operasi dibatalkan
Playlist kosong saat play	Pesan "Playlist kosong." ditampilkan
File MP3 tidak ada saat load	Try-Except menangkap exception, fallback ke nama file
Metadata ID3 tidak lengkap	Gunakan nilai default "Unknown" dan fallback parsing nama file
File CSV tidak ada	Program membuat file CSV baru saat save pertama kali

contoh error handling:

```
1 # Validasi judul dan artis
2 if title.strip() == "" or artist.strip() == "":
3     print("Judul dan artis tidak boleh kosong!")
4     return
5
6 # Validasi ID numeric
7 if not id_song.isdigit():
8     print("ID harus angka!")
9     continue
10
11 # Try-Except untuk ekstraksi metadata
12 try:
13     audio = MP3(full_path, ID3=EasyID3)
14     if 'title' in audio:
15         title = audio['title'][0]
16     if 'artist' in audio:
17         artist = audio['artist'][0]
18 except:
19     # Fallback ke nama file
20     if "-" in filename:
21         artist, title = filename.split("-", 1)
22
23 # Validasi mode
24 if mode in ['semua', 'satu', 'off']:
25     self.loop_mode = mode
26 else:
27     print("Mode tidak valid!")
```

Gambar 3.10 Error Handling

BAB IV

PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

4.1.1 Hasil Pengujian Fungsionalitas CRUD

Create (Penambahan Lagu) berdasarkan hasil pengujian, fitur penambahan lagu telah berjalan sesuai dengan rancangan sistem. Lagu dapat ditambahkan secara manual melalui file picker, kemudian sistem mampu melakukan ekstraksi metadata MP3 berupa judul, artis, dan durasi secara otomatis. Mekanisme fallback juga berhasil diterapkan untuk menangani file yang tidak memiliki ID3 tag dengan mengambil informasi alternatif dari nama file. Selain itu, sistem memiliki validasi untuk mencegah terjadinya duplikasi ID lagu. Proses penambahan node baru dilakukan pada bagian tail Double Linked List dengan kompleksitas $O(1)$, setelah proses pengecekan duplikasi yang membutuhkan kompleksitas $O(n)$.

Read (Menampilkan Data Lagu) fitur penampilan playlist berhasil menampilkan seluruh data lagu dengan format yang terstruktur dan mudah dipahami. Proses traversal dilakukan menggunakan struktur data Double Linked List dari head menuju tail tanpa mengalami kehilangan maupun pengulangan data. Kompleksitas proses pembacaan data sesuai dengan teori yaitu $O(n)$, karena seluruh node perlu dikunjungi untuk menampilkan isi playlist.

Update (Perubahan Data Lagu) fitur edit lagu berhasil melakukan perubahan data secara parsial, yaitu hanya atribut yang diberikan oleh pengguna yang akan diperbarui. Perubahan data dilakukan langsung pada node yang tersimpan di memori tanpa perlu memindahkan posisi node dalam Double Linked List. Struktur pointer prev dan next tetap terjaga setelah proses pembaruan dilakukan. Validasi ID juga berhasil memastikan bahwa pengguna hanya dapat melakukan perubahan terhadap data lagu yang tersedia.

Delete (Penghapusan Data Lagu) pengujian fitur penghapusan menunjukkan bahwa sistem mampu menghapus node pada posisi head, tail, maupun bagian tengah playlist dengan baik. Setelah proses penghapusan dilakukan, hubungan pointer prev dan next berhasil diperbarui sehingga struktur Double Linked List tetap berjalan dengan benar. Apabila lagu yang sedang diputar dihapus, sistem akan menghentikan pemutaran dan melakukan reset terhadap variabel current. Kompleksitas penghapusan adalah $O(n)$ karena membutuhkan pencarian node terlebih dahulu.

4.1.2 Hasil Pengujian Fitur Playback

Play, Next, dan Previous hasil pengujian menunjukkan bahwa library pygame.mixer berhasil digunakan untuk melakukan pemutaran file MP3. Navigasi perpindahan lagu menggunakan pointer pada Double Linked List dapat berjalan dengan kompleksitas $O(1)$. Mode shuffle berhasil diterapkan dengan cara mengumpulkan seluruh node playlist kemudian memilih lagu secara acak dengan kompleksitas $O(n)$. Selain itu, fitur loop dengan pilihan semua lagu, satu lagu, maupun nonaktif telah berjalan sesuai dengan spesifikasi. Setiap lagu yang diputar juga berhasil tercatat ke dalam struktur Stack sebagai riwayat pemutaran.

Pause, Resume, dan Stop kontrol status pemutaran musik berjalan dengan baik. Sistem mampu melakukan pause tanpa menyebabkan error, kemudian fitur resume dapat melanjutkan musik dari posisi terakhir sebelum dihentikan. Fungsi stop juga dapat menghentikan pemutaran sesuai dengan perintah pengguna.

4.1.3 Hasil Pengujian Searching dan Sorting

Sequential Search (Berdasarkan Judul dan Artis)

Fitur pencarian menggunakan metode sequential search berhasil menemukan data lagu berdasarkan judul maupun nama artis. Sistem mendukung pencarian tanpa memperhatikan huruf besar atau kecil (case-insensitive) serta menerapkan substring matching sehingga pengguna tidak harus memasukkan kata pencarian secara lengkap. Kompleksitas pencarian adalah $O(n)$, yang masih sesuai untuk ukuran dataset kecil hingga menengah.

Bubble Sort

Pengujian fitur pengurutan menunjukkan bahwa algoritma bubble sort berhasil mengurutkan lagu berdasarkan judul secara alfabetis dan berdasarkan durasi lagu. Proses sorting durasi dilakukan dengan mengubah format waktu menjadi satuan detik agar dapat dibandingkan. Setelah proses pertukaran data dilakukan, struktur pointer head, tail, prev, dan next tetap berada dalam kondisi yang benar. Kompleksitas $O(n^2)$ masih dapat diterima untuk jumlah playlist dengan kapasitas sekitar 100 lagu.

4.1.4 Hasil Pengujian History Stack

Hasil pengujian menunjukkan bahwa setiap lagu yang diputar berhasil dimasukkan ke dalam Stack menggunakan prinsip LIFO (Last In First Out). Fitur tampilan riwayat mampu menampilkan daftar lagu berdasarkan urutan terbaru diputar terlebih dahulu. Operasi pop dapat menghapus data riwayat dengan benar. Pembatasan maksimal 100 data berhasil diterapkan untuk menghindari penggunaan memori yang berlebihan. Struktur riwayat pemutaran juga tetap terpisah dari playlist utama sehingga tidak memengaruhi Double Linked List.

4.1.5 Hasil Pengujian File Handling dan Persistensi Data

Sistem berhasil melakukan pemindaian folder Music ketika program pertama kali dijalankan. Data playlist dapat disimpan menggunakan format CSV dengan encoding UTF-8 sehingga mendukung karakter khusus. Setelah program ditutup dan dibuka kembali, data sebelumnya tetap tersedia karena proses penyimpanan dan pemuatan data berjalan dengan baik. Format CSV juga mudah dibaca serta dapat dilakukan perubahan secara manual apabila diperlukan. Mekanisme error handling berhasil mencegah program mengalami crash ketika terjadi masalah pada proses operasi file.

4.2 Kelebihan Sistem

4.2.1 Arsitektur Struktur Data

Double Linked List sebagai Struktur Playlist.

Penggunaan Double Linked List memberikan kemampuan navigasi dua arah, yaitu perpindahan lagu ke depan maupun ke belakang dengan kompleksitas $O(1)$. Proses penghapusan node dapat dilakukan tanpa perlu menggeser elemen lain seperti pada array sehingga lebih efisien. Struktur ini juga menggunakan alokasi memori dinamis sehingga tidak membutuhkan ukuran awal tertentu. Oleh karena itu, Double Linked List sesuai diterapkan pada aplikasi pemutar musik yang membutuhkan navigasi playlist secara fleksibel.

Stack sebagai Struktur Riwayat Pemutaran

Struktur Stack digunakan untuk menyimpan riwayat lagu yang telah diputar karena menerapkan prinsip LIFO yang sesuai dengan kebutuhan pengguna dalam mengakses lagu terakhir. Operasi push dan pop dapat dilakukan secara efisien dengan kompleksitas $O(1)$, sedangkan proses melihat seluruh isi history membutuhkan $O(n)$. Pembatasan jumlah data juga diterapkan untuk menjaga penggunaan memori. Selain itu, Stack dibuat terpisah dari playlist

sehingga tidak mengganggu struktur utama Double Linked List.

4.2.2 Fitur dan Fungsionalitas Sistem

1. Implementasi CRUD Lengkap

Sistem telah menyediakan fitur pengelolaan data lagu yang meliputi penambahan lagu dengan ekstraksi metadata otomatis, penampilan seluruh playlist, perubahan data secara parsial, serta penghapusan lagu dengan pengelolaan pointer yang sesuai.

2. Kontrol Pemutaran Musik Real-time

Aplikasi mendukung fitur pemutaran musik seperti play, pause, resume, dan stop menggunakan pygame.mixer. Navigasi next dan previous memanfaatkan struktur Double Linked List dengan kompleksitas $O(1)$. Selain itu, tersedia fitur shuffle serta loop yang dapat disesuaikan dengan kebutuhan pengguna.

3. Pencarian Data yang Fleksibel

Fitur pencarian mendukung pencocokan tanpa memperhatikan kapitalisasi huruf serta pencarian berdasarkan sebagian kata. Pengguna dapat mencari lagu berdasarkan judul maupun artis, kemudian hasil pencarian akan dikembalikan dalam bentuk daftar pilihan.

4. Pengurutan Data Lagu

Sistem menyediakan fitur pengurutan berdasarkan judul secara alfabetis dan berdasarkan durasi lagu. Setelah proses sorting dilakukan, struktur pointer pada Double Linked List tetap dalam kondisi konsisten.

5. Penyimpanan Data Persisten

Data playlist dapat disimpan dalam file CSV dengan format UTF-8. Sistem akan memuat kembali data secara otomatis ketika program dijalankan sehingga informasi lagu tidak hilang setelah program ditutup. File CSV juga dapat digunakan sebagai media backup maupun pengeditan manual.

4.2.3 User Experience

1. Tampilan Menu yang Terstruktur

Sistem memiliki menu yang tersusun secara jelas melalui pembagian submenu seperti CRUD, pengelolaan lagu, dan utility. Setiap pilihan menu memiliki deskripsi yang mudah dipahami sehingga pengguna dapat mengoperasikan aplikasi dengan lebih mudah.

2. Error Handling yang Baik

Validasi input diterapkan untuk mengurangi kemungkinan kesalahan pengguna. Selain itu, penggunaan try-except pada proses operasi file serta mekanisme fallback metadata membantu sistem tetap berjalan ketika terjadi kondisi yang tidak sesuai.

3. Fleksibilitas Ekstraksi Metadata

Sistem mampu mengambil informasi lagu melalui ID3 tag apabila tersedia. Jika metadata tidak ditemukan, sistem akan mencoba membaca informasi dari format nama file seperti "Artist - Title". Apabila proses gagal, sistem akan memberikan nilai default "Unknown" dan pengguna tetap dapat melakukan perubahan metadata secara manual.

4.3 Keterbatasan Sistem

4.3.1 Aspek Teknis

1. Antarmuka Berbasis CLI

Sistem masih menggunakan Command Line Interface sehingga pengguna harus memasukkan perintah melalui terminal. Pendekatan ini membuat aplikasi kurang sesuai bagi pengguna yang lebih terbiasa menggunakan antarmuka grafis berbasis mouse atau layar sentuh.

2. Keterbatasan Kompleksitas Algoritma

Metode sequential search memiliki kompleksitas $O(n)$ sehingga kurang optimal untuk jumlah playlist yang sangat besar. Selain itu, penggunaan bubble sort dengan kompleksitas $O(n^2)$ membuat proses pengurutan kurang efisien pada dataset besar karena belum menggunakan algoritma sorting yang lebih optimal.

3. Struktur Penyimpanan Data

Penyimpanan menggunakan file CSV masih memiliki keterbatasan karena hanya berbasis teks dan tidak memiliki fitur keamanan seperti enkripsi. Sistem juga belum mendukung penggunaan database relasional sehingga belum memiliki kemampuan query optimization maupun transaction support.

4. Dukungan Format Audio

Sistem hanya mendukung file dengan format MP3. Library pygame.mixer memiliki

keterbatasan dibandingkan audio engine profesional sehingga fitur seperti equalizer, efek suara, dan pemrosesan audio lanjutan belum tersedia.

4.4.2 Aspek Fungsional

1. Keterbatasan Multi-user

Aplikasi dirancang hanya untuk penggunaan satu pengguna. Sistem belum memiliki fitur autentikasi, pengaturan hak akses, maupun dukungan penggunaan secara bersamaan oleh beberapa pengguna.

2. Keterbatasan Networking

Sistem belum mendukung layanan streaming berbasis internet maupun sinkronisasi dengan platform musik lain. Aplikasi hanya dapat digunakan secara offline dengan sumber musik lokal.

3. Keterbatasan Metadata

Metadata yang dapat diekstraksi masih terbatas pada informasi judul, artis, dan durasi. Data tambahan seperti genre, tahun rilis, album, cover art, dan lirik belum tersedia. Selain itu, sistem belum mendukung perubahan metadata secara massal.

4. Fitur Tambahan yang Belum Dikembangkan

Beberapa fitur lanjutan seperti playlist favorit, kategori lagu, sistem rating, rekomendasi berdasarkan riwayat pemutaran, tampilan lirik, dan mode karaoke belum tersedia dalam sistem.

4.4.3 Batasan Performa

1. Skalabilitas Sistem

Proses traversal pada Double Linked List memiliki kompleksitas $O(n)$, sehingga dapat menjadi hambatan ketika jumlah lagu dalam playlist semakin besar. Penggunaan bubble sort dengan kompleksitas $O(n^2)$ juga menyebabkan performa menurun pada dataset besar. Selain itu, kebutuhan memori meningkat secara linear mengikuti jumlah lagu yang tersimpan.

2. Responsivitas Aplikasi

Pada playlist dengan jumlah lagu yang sangat banyak, proses sorting dapat menyebabkan keterlambatan respons aplikasi. Mode shuffle juga membutuhkan waktu tambahan karena harus mengumpulkan seluruh node terlebih dahulu.

BAB V. KENDALA DAN SOLUSI

Berikut adalah ringkasan kendala yang dihadapi selama pengembangan beserta solusinya:

Tabel 5.1 Kendala dan Solusi

Kendala	Solusi
NoneType Exception saat navigasi DLL di ujung playlist atau playlist kosong	Menambahkan validasi pointer yang ketat. Jika mode Loop='semua' aktif dan pointer berada di tail, fungsi next_song() secara otomatis mengembalikan pointer ke head.
Crash saat membaca metadata ID3 dari file MP3 yang tidak lengkap	Membungkus blok pembacaan audio dalam Try-Except. Jika metadata tidak ditemukan, nilai fallback 'Unknown' dikembalikan. Ditambahkan logika parsing nama file sebagai fallback.
Data playlist hilang ketika program ditutup	Mengimplementasikan file handling menggunakan format CSV (playlist.csv) untuk persistensi data. Data otomatis dimuat saat program dibuka dan disimpan setiap ada perubahan.
Konflik pointer saat menggabungkan fitur Shuffle dengan DLL	Menambahkan Stack terpisah untuk mencatat urutan pemutaran dalam mode Shuffle, sehingga riwayat tetap dapat diakses tanpa merusak struktur DLL utama.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Berikut adalah rincian pembagian tugas masing-masing anggota kelompok selama pengerjaan proyek:

Nama	NIM	Tugas
Pratama Fahriel Sanjaya	J0403251053	• Perancangan arsitektur sistem (Double Linked List dan Stack) • Implementasi modul Playlist_Musik.py (program utama + DLL) • Implementasi modul history_stack.py (Stack riwayat pemutaran) • Implementasi modul search_sort.py (Sequential Search + Bubble Sort) • Implementasi modul data_handler.py (baca/tulis CSV) • Penulisan laporan BAB I, II, V • Pembuatan flowchart dan dokumentasi teknis
Muhammad Haris	J0403251141	• Implementasi fitur CRUD (tambah, hapus, perbarui lagu) di Playlist_Musik.py • Ekstraksi metadata MP3 dan error handling (Try-Except) • Penulisan laporan BAB III (Implementasi) dan BAB IV (Pengujian)
Rony Wijaya	J0403251047	• Implementasi modul Playlist_Musik.py (program utama + DLL) • Implementasi modul Playlist_Musik.py (Play, Pause, Stop, Resume lagu) • Pengujian sistem dan validasi input/error handling • Penulisan laporan BAB , VI, VII • Pengelolaan repositori GitHub dan Readme.md

Tabel 6.1 Pembagian Tugas

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian Aplikasi Playlist Musik Berbasis Terminal yang telah dilakukan oleh Kelompok 13, dapat ditarik beberapa kesimpulan sebagai berikut.

1. Aplikasi Playlist Musik Berbasis Terminal berhasil dikembangkan menggunakan bahasa pemrograman Python dengan antarmuka *Command Line Interface* (CLI). Aplikasi ini mampu menjalankan fungsi-fungsi utama sebuah pemutar musik, meliputi pemutaran audio nyata menggunakan library `pygame.mixer`, manajemen playlist lengkap dengan fitur CRUD, serta mode *Shuffle* dan *Loop*.
2. Implementasi struktur data *Double Linked List* terbukti tepat dan efisien sebagai fondasi utama aplikasi. Navigasi antar lagu (*Next/Previous*) dapat dilakukan dalam kompleksitas waktu $O(1)$, dan operasi penambahan serta penghapusan lagu berjalan tanpa *overhead* pergeseran elemen seperti yang terjadi pada struktur Array. Pemilihan ini selaras dengan teori yang dikemukakan Goodrich, Tamassia, dan Goldwasser (2013) mengenai kesesuaian *Linked List* untuk data yang bersifat dinamis dan membutuhkan traversal sekuensial.
3. Implementasi Stack untuk manajemen riwayat pemutaran berhasil memodelkan perilaku *Playback History* secara akurat melalui prinsip LIFO. Pengguna dapat mengakses kembali lagu terakhir yang diputar dalam waktu $O(1)$, bahkan ketika mode *Shuffle* aktif. Penggunaan library mutagen untuk ekstraksi metadata .mp3 juga berhasil meningkatkan pengalaman pengguna dengan mengotomatisasi pengisian informasi judul, artis, dan durasi lagu.
4. Fitur pencarian menggunakan *Sequential Search* melalui traversal DLL dan fitur pengurutan menggunakan *Bubble Sort* berhasil diimplementasikan dan berjalan dengan baik. Persistensi data menggunakan file CSV memastikan data playlist tetap tersimpan dan dapat dimuat kembali setiap kali program dijalankan.

Saran Pengembangan

Meskipun aplikasi ini telah berhasil memenuhi seluruh tujuan yang ditetapkan, terdapat beberapa aspek yang dapat dikembangkan lebih lanjut pada iterasi berikutnya, antara lain:

1. Pengembangan Antarmuka: Aplikasi dapat dikembangkan dengan antarmuka grafis (GUI) menggunakan *library* seperti Tkinter atau PyQt untuk meningkatkan kemudahan penggunaan dan pengalaman pengguna secara signifikan.
2. Peningkatan Algoritma Pencarian: *Sequential Search* yang digunakan saat ini memiliki kompleksitas $O(n)$. Untuk dataset yang lebih besar, dapat dipertimbangkan implementasi *Binary Search* (dengan terlebih dahulu mengurutkan data) atau penggunaan struktur data Hash Table untuk mencapai kompleksitas $O(1)$ rata-rata.
3. Peningkatan Algoritma Pengurutan: *Bubble Sort* dengan kompleksitas $O(n^2)$ dapat digantikan dengan algoritma yang lebih efisien seperti Merge Sort ($O(n \log n)$) untuk menangani koleksi musik yang lebih besar.
4. Dukungan Database: Penyimpanan data saat ini menggunakan file CSV. Penggunaan database seperti SQLite akan meningkatkan keamanan data, mendukung *query* yang lebih kompleks, serta membuka kemungkinan fitur multi user.
5. Fitur Playlist Ganda: Sistem dapat dikembangkan untuk mendukung pengelolaan lebih dari satu playlist sekaligus, misalnya dengan menggunakan struktur data *Tree* atau *Graph* untuk merepresentasikan hierarki dan relasi antar playlist.
6. Integrasi Streaming Online: Aplikasi dapat dikembangkan untuk mendukung pemutaran lagu dari sumber online, misalnya melalui integrasi dengan API layanan streaming musik.

DAFTAR PUSTAKA

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, Data Structures and Algorithms in Python. Hoboken, NJ, USA: John Wiley & Sons, 2013.
- B. N. Miller dan D. L. Ranum, Problem Solving with Algorithms and Data Structures Using Python, 2nd ed. Portland, OR, USA: Franklin, Beedle & Associates, 2011.
- R. D. Setiyawan, D. Hermawan, dan A. F. Abdillah, “Penggunaan Struktur Data Stack dalam Pemrograman C++ dengan Pendekatan Array dan Linked List,” Jurnal Teknologi Informasi dan Komputer, 2024.
- S. S. Winarsih dan A. W. Wahono, “Implementasi dan Pengujian Struktur Data Berbasis Acuan untuk Program Aplikasi,” Jurnal TIKomSIN, vol. 10, no. 2, hal. 11–17, 2022.
- O. Wardhani dan I. Alfath, Modul Praktikum Struktur Data: Stack. Indonesia: 2024.
- Departemen Teknik Elektro Universitas Negeri Malang, “Modul 6: Single & Double Linked List,” Modul Praktikum Algoritma dan Struktur Data. Malang: UM, 2016.

LAMPIRAN

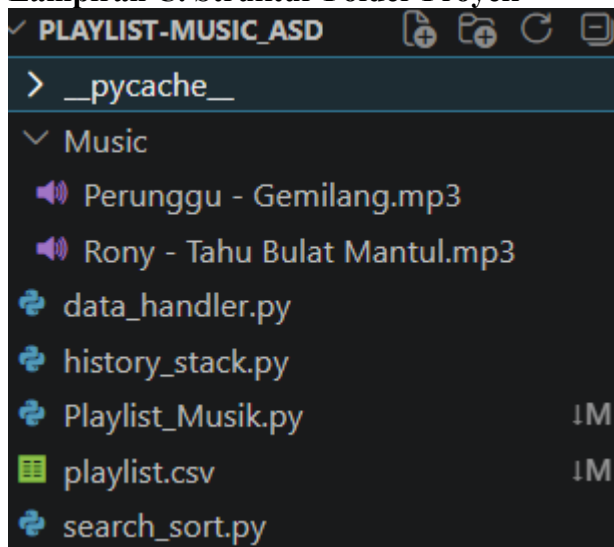
Lampiran A. Link GitHub

https://github.com/pratamaell/ASD_B_13

Lampiran B. Screenshot Program

```
1  ===== MENU UTAMA =====
2  1. Menu Playlist (CRUD)
3  2. Menu Lagu
4  3. Menu Utility
5  0. Keluar
6
7  Pilih menu:
```

Lampiran C. Struktur Folder Proyek



Lampiran D. Commit History GitHub

Commits on Jun 22, 2026		
Menambahkan Readme.md	SclayGT committed yesterday	410667d
Perbaikan	SclayGT committed yesterday	9448c80
Initial commit	Muh Haris committed yesterday	1e36f08
Commits on May 20, 2026		
menambahkan penambahan lagu	pratamaell committed on May 20	d4ca5dc
debug	pratamaell committed on May 20	82400d2
Perbaikan Menu Edit dan Library Mutagen	SclayGT committed on May 20	dba2368
feat: tambah fitur edit lagu	Muh Haris committed on May 20	f946640
Commits on May 19, 2026		
Resolve merge conflict	Muh Haris committed on May 20	1d7f6d8
Remove pycache	Muh Haris committed on May 20	65769a2
Commits on May 14, 2026		
menghapus beberpa fungsi	pratamaell committed on May 19	b538a50
Commits on May 13, 2026		
Perbaikan Menu Tambah Lagu dan CSV	SclayGT committed on May 14	fd3e7fc
Perbaikan Menu Tambah Lagu dan CSV	SclayGT committed on May 14	6b988d6
Commits on Apr 3, 2026		
memperbaiki bug	pratamaell committed on May 13	743c093
Commits on Mar 30, 2026		
File Handling, Sort, Search ,Menu Utama	pratamaell committed on Apr 3	4d1c05c
Commits on Mar 7, 2026		
Play lagu + Next / Previous / Looping	SclayGT committed on Mar 30	7b03026
upload playlist musik	Muh Haris committed on Mar 7	c21f809
© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information		

Lampiran E. Source Code Utama

```
# =====
# PROGRAM UTAMA
# =====

playlist = Playlist()
playlist.load_music_folder()

print("Playlist Musik siap! Gunakan menu untuk navigasi.")

def menu_playlist():
    while True:
        print("\n--- MENU PLAYLIST (CRUD) ---")
        print("1. Tambah Lagu")
        print("2. Lihat Playlist")
        print("3. Hapus Lagu")
        print("4. Edit Lagu")
        print("0. Kembali ke Menu Utama")

        pilihan = input("Pilih menu: ").strip()

        if pilihan == "1":
            print("\nMembuka file picker...")
            file_path = pick_music_file()

            if file_path is None:
                print("Pembatalan pemilihan file.")
                continue

            if not os.path.exists(file_path):
                print("File tidak ditemukan!")
                continue

            file_name = os.path.basename(file_path)
            filename = os.path.splitext(file_name)[0]

            title = filename
```

```

artist = "Unknown"
duration = "Unknown"

try:
    audio = MP3(file_path, ID3=EasyID3)

    # title
    if 'title' in audio:
        title = audio['title'][0]

    # artist
    if 'artist' in audio:
        artist = audio['artist'][0]

    # duration
    audio_info = MP3(file_path)

    total_seconds = int(audio_info.info.length)

    minutes = total_seconds // 60
    seconds = total_seconds % 60

    duration = f"{minutes}:{seconds:02d}"

except:
    pass

# fallback nama file
if artist == "Unknown":
    if "-" in filename:
        artist, title = filename.split("-", 1)

# auto ID random
id_song = str(random.randint(1000, 9999))

playlist.add_song(

```

```

        id_song,
        title,
        artist,
        duration,
        file_path
    )

elif pilihan == "2":
    playlist.show_playlist()

elif pilihan == "3":
    id_song = input("ID lagu yang dihapus: ").strip()
    if not id_song.isdigit():
        print("ID harus angka!")
        continue
    playlist.delete_song(id_song)

# =====
# BLOK EDIT LAGU DITAMBAHKAN
# =====
elif pilihan == "4":
    id_song = input("Masukkan ID lagu yang ingin diedit: ").strip()

    print("Tips: Kosongkan isian dan langsung tekan Enter jika tidak ingin
mengubah data tersebut.")
    new_title = input("Judul baru: ").strip()
    new_artist = input("Artis baru: ").strip()
    new_duration = input("Durasi baru (contoh 03:45): ").strip()
    new_file_path = input("Path file MP3 baru: ").strip()

    playlist.edit_song(id_song, new_title, new_artist, new_duration,
new_file_path)

elif pilihan == "5":
    save_to_csv(playlist) # Anggota 3: Simpan data playlist ke file CSV untuk
persistensi

```

```

        elif pilihan == "0":
            break

        else:
            print("Menu tidak valid!")

def menu_lagu():
    while True:
        print("\n--- MENU LAGU ---")
        print("1. Play Lagu")
        print("2. Next Lagu")
        print("3. Previous Lagu")
        print("4. Pause Lagu")
        print("5. Resume Lagu")
        print("6. Stop Lagu")
        print("0. Kembali ke Menu Utama")

        pilihan = input("Pilih menu: ").strip()
        if pilihan == "1":
            playlist.play_song()

        elif pilihan == "2":
            playlist.next_song()

        elif pilihan == "3":
            playlist.previous_song()

        elif pilihan == "4":
            playlist.pause_song()

        elif pilihan == "5":
            playlist.resume_song()

        elif pilihan == "6":
            playlist.stop_song()

```

```

elif pilihan == "0":
    break

else:
    print("Menu tidak valid!")

def menu_utility():
    while True:
        print("\n--- MENU UTILITY ---")
        print("1. Lihat Riwayat")
        print("2. Cari berdasarkan Judul")
        print("3. Cari berdasarkan Artis")
        print("4. Urutkan berdasarkan Alfabet")
        print("5. Urutkan berdasarkan Durasi Pendek-Panjang")
        print("6. Simpan ke CSV")
        print("0. Kembali ke Menu Utama")

        pilihan = input("Pilih menu: ").strip()
        if pilihan == "1":
            playlist.history.show_history()

        elif pilihan == "2":
            keyword = input("Kata kunci judul: ").strip()
            results = search_title(playlist, keyword)
            if results:
                print(f"\nHasil pencarian untuk '{keyword}':")
                for song in results:
                    print(f"{song.id}. {song.title} - {song.artist}")
            else:
                print("Tidak ditemukan.")

        elif pilihan == "3":
            keyword = input("Kata kunci artis: ").strip()
            results = search_artist(playlist, keyword)

```



```

        if results:
            print(f"\nHasil pencarian untuk '{keyword}':")
            for song in results:
                print(f"{song.id}. {song.title} - {song.artist}")
            else:
                print("Tidak ditemukan.")

    elif pilihan == "4":
        sort_title_asc(playlist)

    elif pilihan == "5":
        sort_duration_asc(playlist)

    elif pilihan == "6":
        save_to_csv(playlist)

    elif pilihan == "0":
        break

    else:
        print("Menu tidak valid!")

while True:
    print("\n===== MENU UTAMA =====")
    print("1. Menu Playlist (CRUD)")
    print("2. Menu Lagu")
    print("3. Menu Utility")
    print("0. Keluar")

    pilihan = input("Pilih menu: ").strip()
    if pilihan == "1":
        menu_playlist()
    elif pilihan == "2":
        menu_lagu()
    elif pilihan == "3":
        menu_utility()

```

```
elif pilihan == "0":  
    print("Program selesai.")  
    break  
else:  
    print("Menu tidak valid!")
```